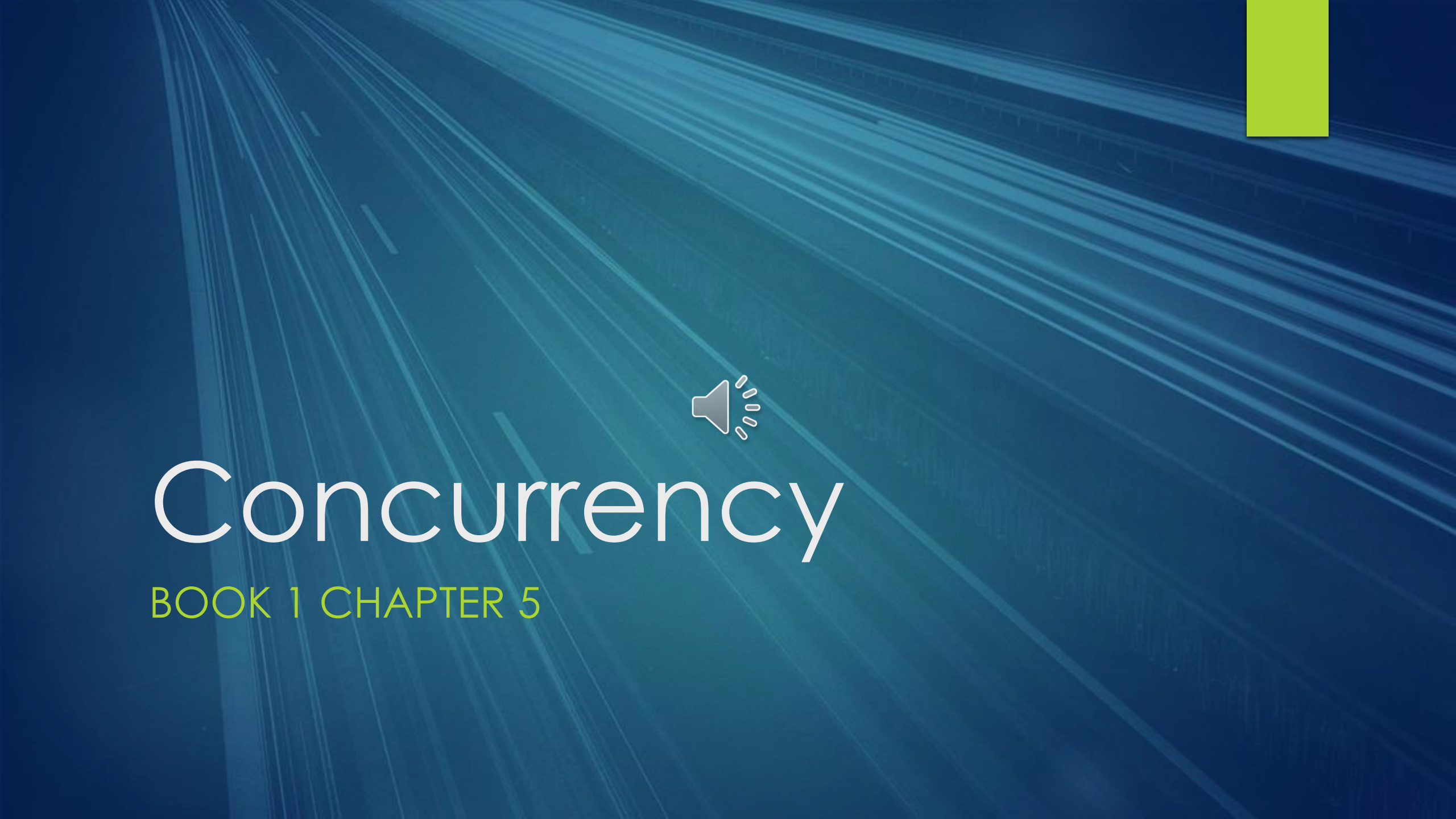
CS-317 Operating Systems

LECTURE 14



Concurrency

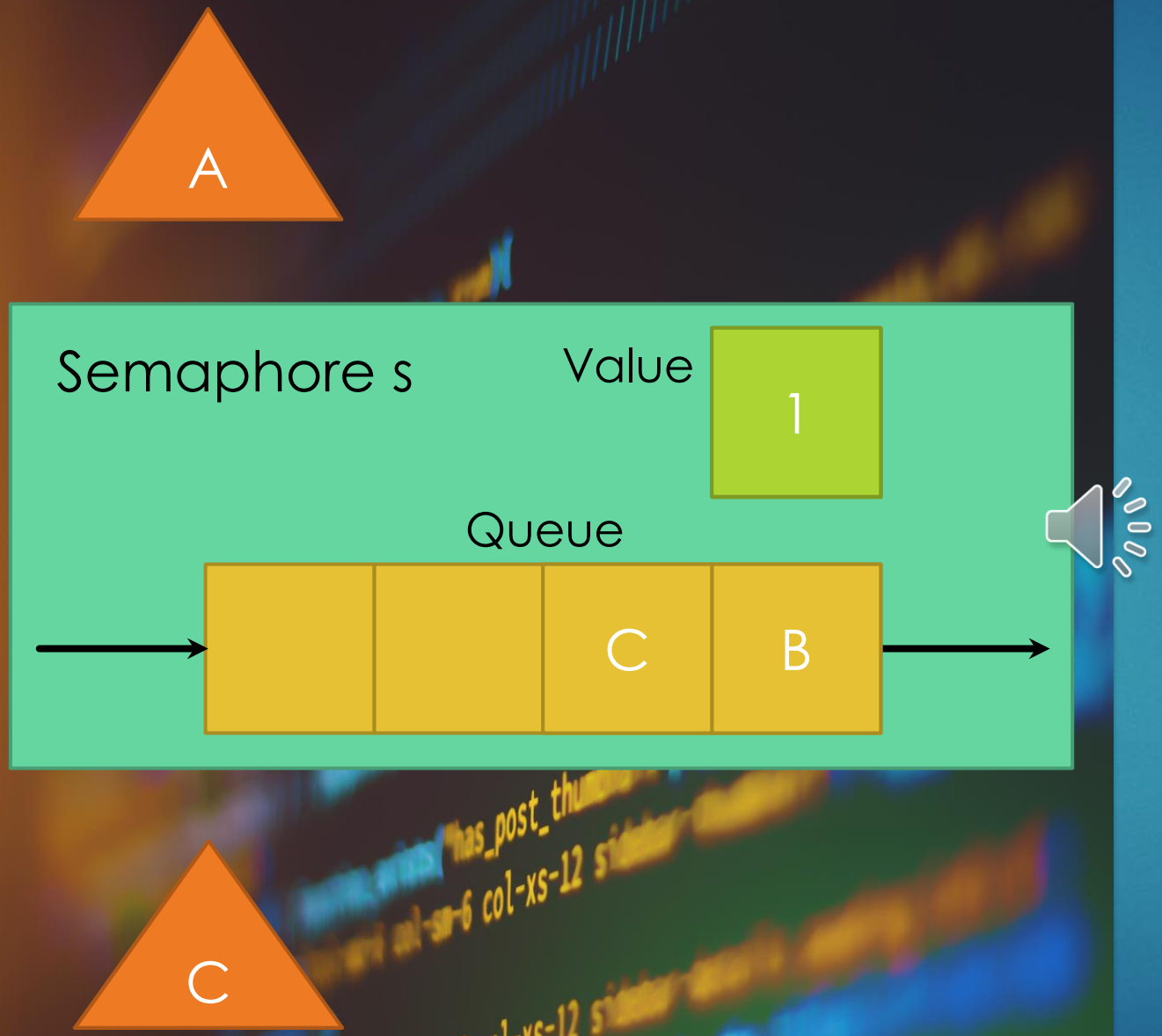
BOOK 1 CHAPTER 5

In the last lecture…

- We looked in detail at the compare and swap instruction.
- We learned how to trace the execution of parallel processes.
- We looked in detail at the exchange instruction.
- We conducted an analysis of the machine-instruction approach to mutual exclusion.
- We learned about semaphores.

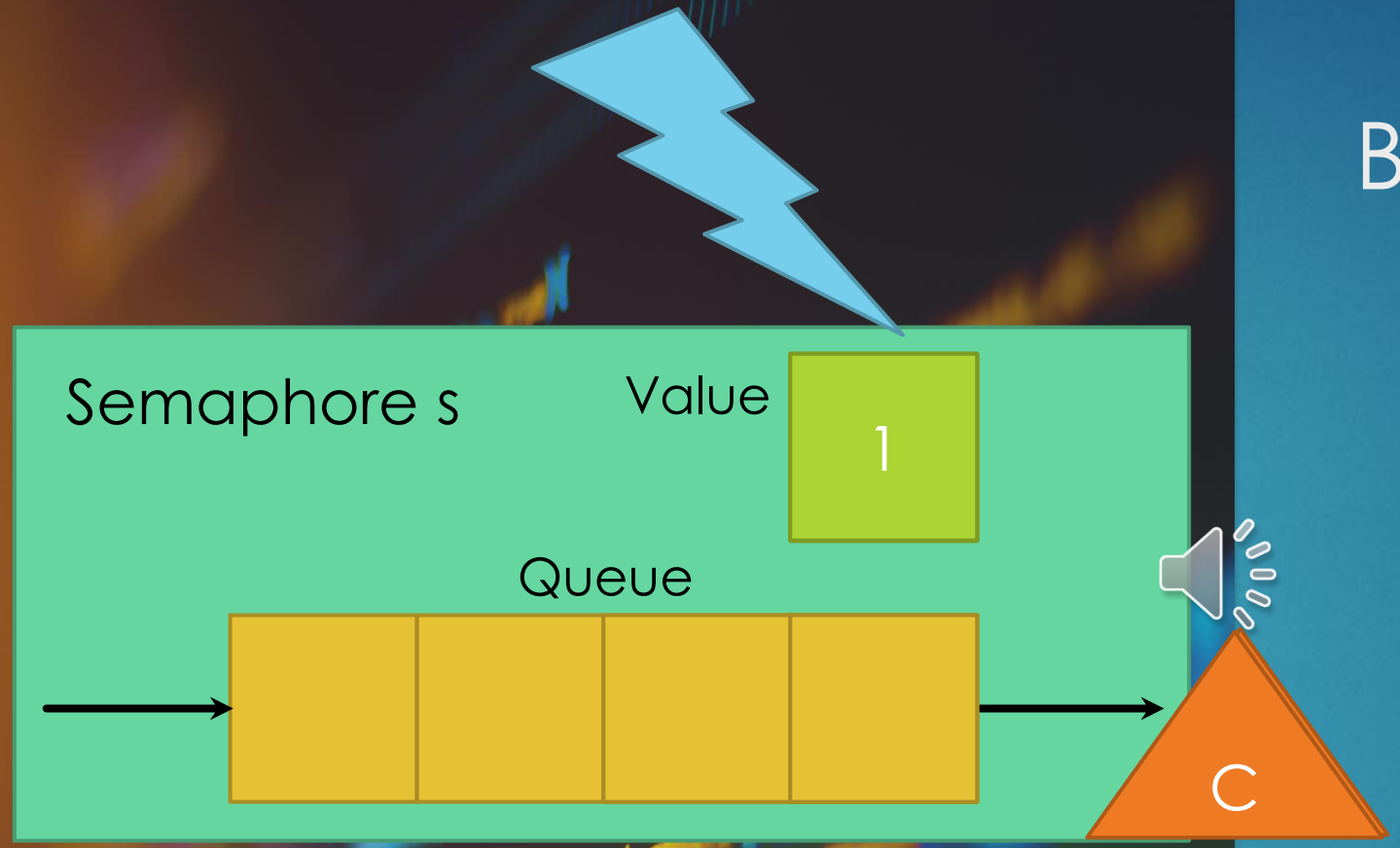
Semaphores

CONCEPT, USE AND IMPLEMENTATION



Binary semaphore

- A binary semaphore can be defined by the following three operations:
  1. A binary semaphore may be initialized to 0 or 1.
  2. If the value of a binary semaphore is zero, then the process executing the semWaitB is blocked. If the value is one, then the value is changed to zero and the process continues execution.



Binary semaphore

3. The semSignalB operation checks if the queue of the binary semaphore is empty. If so, its value is set to one. Otherwise, a process is unblocked from the queue and added to the pool of executable processes.

```
struct binary_semaphore {
        enum {zero, one} value;
        queueType queue;
};
void semWaitB(binary_semaphore s)
{
        if (s.value == one)
             s.value = zero;
        else {
                    /* place this process in s.queue */;
                    /* block this process */;
        }
}
void semSignalB(semaphore s)
{
        if (s.queue is empty())
             s.value = one;
        else {
                    /* remove a process P from s.queue */;
                    /* place process P on ready list */;
        }
}
```

the process enters the critical section

resource free again

Figure 5.4 A Definition of Binary Semaphore Primitives

Counting semaphores

To contrast the two types of semaphores, the nonbinary semaphore

is often referred to as either a counting semaphore or a general semaphore.

Mutexes

- You must have noticed that with counting and binary semaphores, a process blocks because it executes the Wait primitive, but it cannot unblock itself.
- Any process blocked on a counting or binary semaphore is unblocked by another process which executes the Signal primitive.
- A concept related to the binary semaphore is the mutex.
- A key difference between the two is that the process that locks the mutex (sets the value to zero) is the one to unlock it (set the value to one).

the two are binary semaphore and mutex

Queue discipline for semaphores

- For both counting semaphores and binary semaphores, a queue is used to hold processes waiting on the semaphore.
- The fairest removal policy is first-in-first-out (FIFO): The process that has been blocked the longest is released from the queue first.
- A semaphore whose definition includes FCFS as the queueing discipline is called a strong semaphore.
- A semaphore that does not specify the order in which processes are removed from the queue is a weak semaphore.

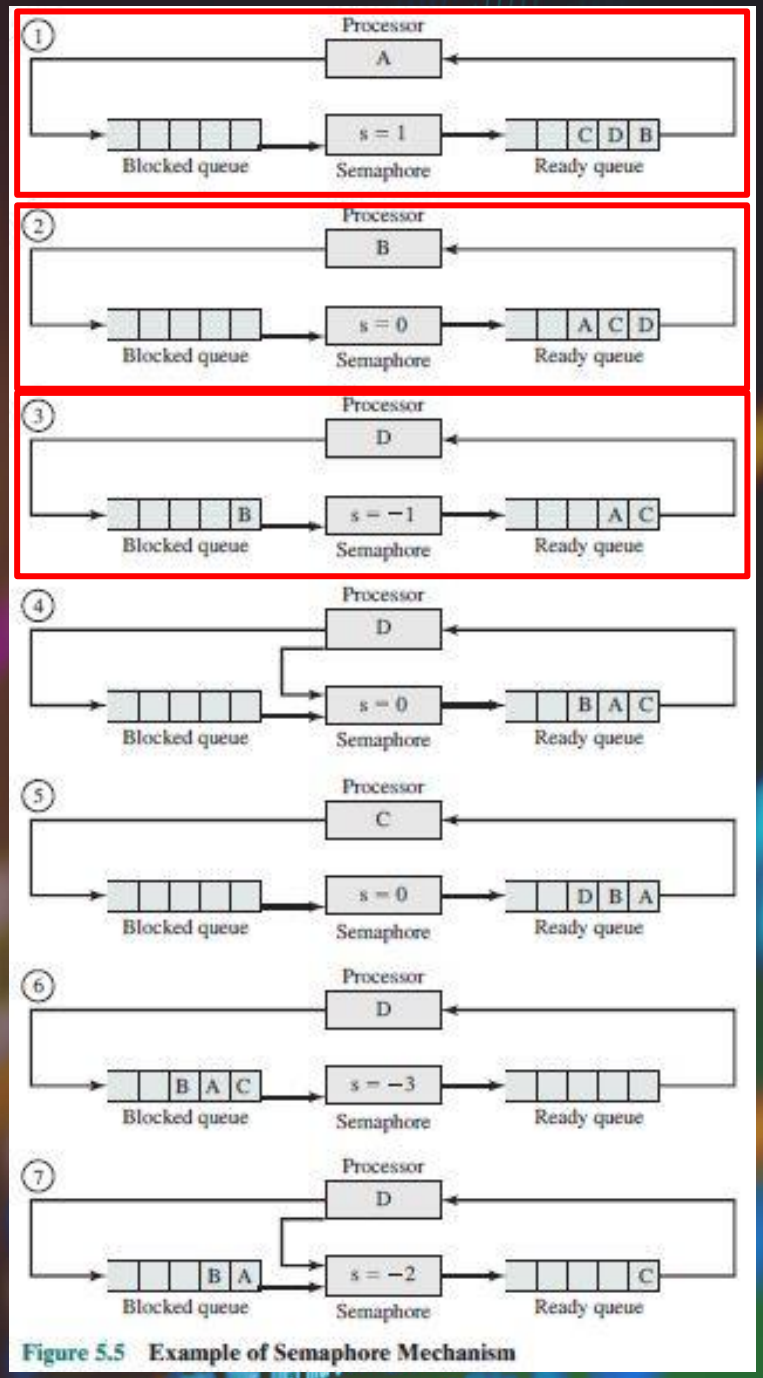


Figure 5.5 Example of Semaphore Mechanism

Operation of strong semaphore

Here processes A, B, and C depend on a result from process D.

1. Initially, A is running, B, D, and C are ready, and the semaphore count is 1, indicating that one of D's results is available. When A issues a semWait(s), s decrements to 0, and A can continue to execute.
2. Subsequently it rejoins the ready queue. Then B runs.
3. B issues a semWait(s) and is blocked. This allows D to run.

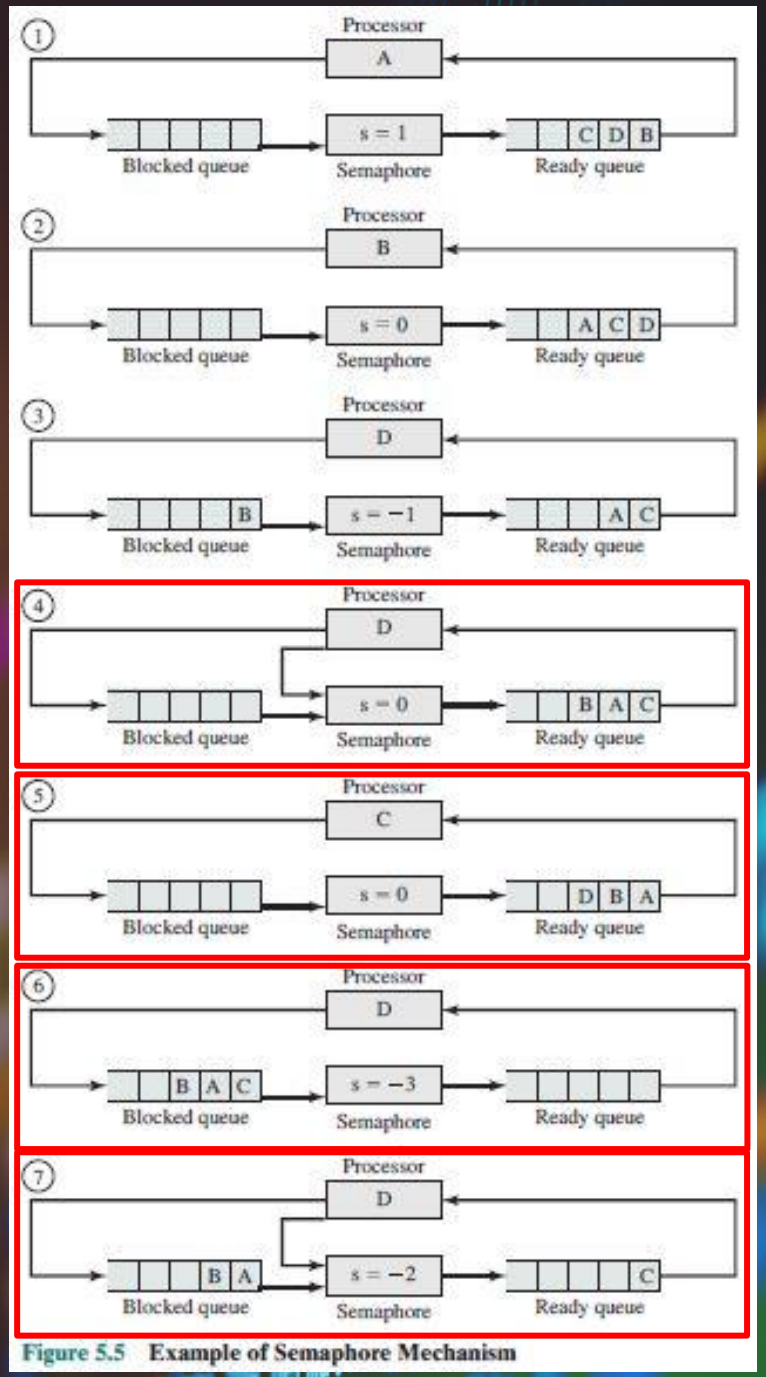


Figure 5.5 Example of Semaphore Mechanism

Operation of strong semaphore

4. When D completes a new result, it issues a semSignal(s), which allows B to move to the ready queue.
5. D rejoins the ready queue and C begins to run.
6. C is blocked when it issues a semWait(s). Similarly, A and B run and are blocked on s. This allows D to resume execution.
7. When D has a result, it issues a semSignal(s), which transfers C to the ready queue.

Later cycles of D will release A and B from the blocked state.

here, D is signaling while other are waitng, hence nta mutex

The power of strong semaphores

- Strong semaphores guarantee freedom from starvation, while weak semaphores do not.

- We will assume strong semaphores in the remaining discussion because:
  - They are more convenient.
  - They are the form of semaphore typically provided by operating systems.

```
/* program mutualexclusion */
const int n = /* number of processes */;
semaphore s = 1;
void P(int i)
{
     while (true) {
          semWait(s);
          /* critical section */;
          semSignal(s);
          /* remainder */;
     }
}
void main()
{
     parbegin (P(1), P(2),…, P(n));
}
```

Figure 5.6 Mutual Exclusion Using Semaphores

```
void P(int i) {
while (true) {
semWait(s);
/*critical section*/;
semSignal(s);
/*remainder*/;
}
}
```

Parallel trace of ME using semaphores

s.count	s.queue ← [←	P1	P2	P3

Lessons from the parallel trace

- ME is enforced, as there is only one process in critical section at any time.
- When a process blocks, it is placed in a queue, where it waits. This is an improvement over the machine-instruction approach because, in that approach, the blocked process busy waits and uses the processor.
- When a process unblocks, it resumes from the next line of code. So after a process moves out of the queue, whenever it is dispatched again, it lands directly into critical section.

ME with semaphores

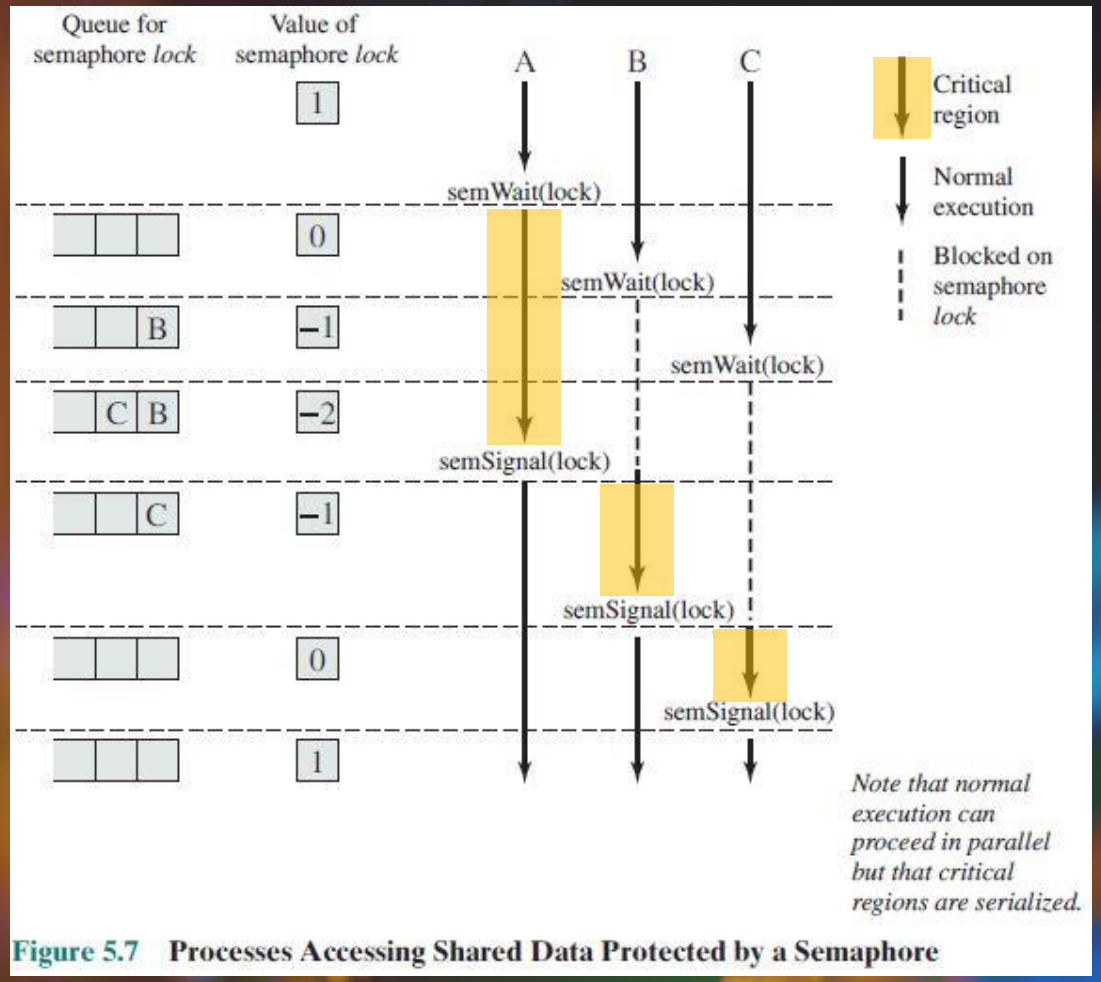


Figure 5.7 Processes Accessing Shared Data Protected by a Semaphore

- This figure shows a possible sequence for three processes A, B and C using the semaphore named lock for ME.
- Process A executes semWait. Because the value is 1, A can immediately enter its critical section and the value decrements to 0.
- While A is in its critical section, both B and C perform a semWait operation each and are blocked in the semaphore's queue.
- When A exits its critical section and performs semSignal, B, which was the first process in the queue, can enter its critical section.

fifo followed

Synchronization with semaphores

```
void A() {
    /*do what A does*/;
}
```

```
void B() {
    /*do what B does*/;
}
```

```
void main() {
    parbegin (A(), B());
}
```

- The following examples will show you how we can use the initialization of semaphores and the placement of semWait and semSignal primitives to enforce any execution order that we desire.

Make A run before B!

```
void A() {
    /*do what A does*/;
}
```

```
void B() {
    /*do what B does*/;
}
```

```
void main() {
    parbegin (A(), B());
}
```

- The entity that runs first must signal that it has executed at the end of its code.
- The entity that runs second must wait for a signal that the first has executed before executing its code.
- The semaphore should be initialized to 0, so that the person who calls semWait blocks.

```
void A() {
    /*do what A does*/;
    semSignal (syn);
}
```

```
void B() {
    semWait (syn);
    /*do what B does*/;
}
```

```
semaphore syn = 0;
void main() {
    parbegin (A(), B());
}
```

Make A run before B!

by not letting A wait there are no chances of A getting blocked since A will not depned on B's signal because only that process requires signal taht was blocked

When A comes first

syn	A	B
0		
	does work	
1	semSignal	
0		semWait
		does work

```
void A() {
    /*do what A does*/;
    semSignal (syn);
}
```

```
void B() {
    semWait (syn);
    /*do what B does*/;
}
```

```
semaphore syn = 0;
void main() {
    parbegin (A(), B());
}
```

Make A run before B!

When B comes first

syn	A	B
0		
-1		semWait (blocks)
	does work	
0	semSignal	unblocks
		does work

```
void A() {
    /*do what A does*/;
    semSignal (syn);
}
```

```
void B() {
    semWait (syn);
    /*do what B does*/;
}
```

```
semaphore syn = 0;
void main() {
    parbegin (A(), B());
}
```

Make A run before B!

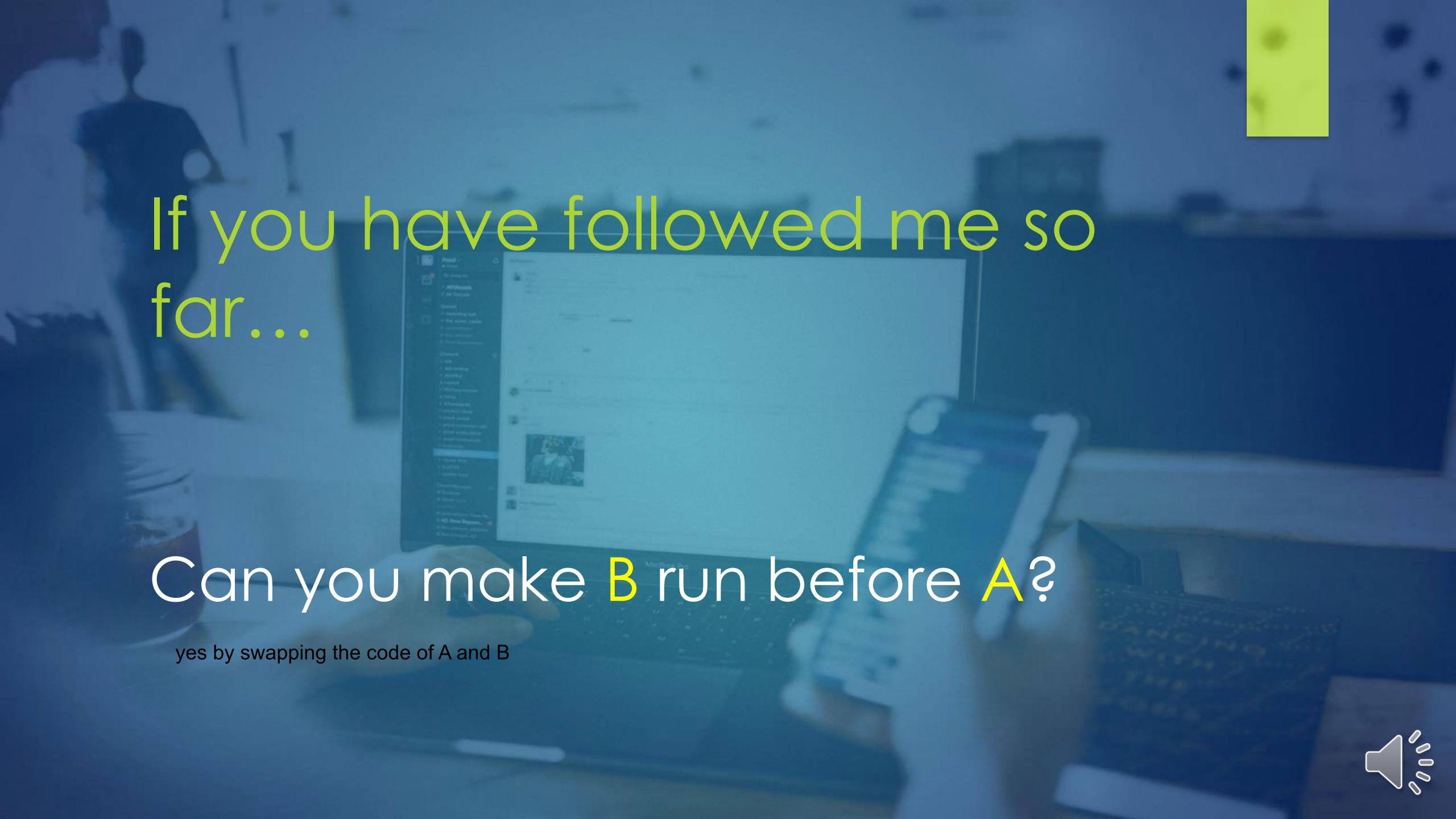
When A and B run in parallel

syn	A	B
0		
-1	does work	semWait (blocks)
0	semSignal	unblocks
		does work

```
void A() {
    /*do what A does*/;
    semSignal (syn);
}
```

```
void B() {
    semWait (syn);
    /*do what B does*/;
}
```

```
semaphore syn = 0;
void main() {
    parbegin (A(), B());
}
```



If you have followed me so far…

Can you make B run before A?

yes by swapping the code of A and B

Make A and B always run together!

```
void A() {
    /*do what A does*/;
}
```

```
void B() {
    /*do what B does*/;
}
```

```
void main() {
    parbegin (A(), B());
}
```

- Each process signals its own arrival and waits for the other.
- We assume two semaphores here, as we require two signals, one to announce the arrival of A, and the other to announce the arrival of B.
- Both semaphores are initialized to 0.

```
void A() {
    semSignal (sA);
    semWait (sB);
    /*do what A does*/;}
```

```
void B() {
    semWait (sA);
    semSignal (sB);
    /*do what B does*/;}
```

```
semaphore sA = 0;
semaphore sB = 0;
void main() {
    parbegin (A(), B());}
```

Make A and B always run together!

When A comes first

sA	sB	A	B
0	0		
1		semSignal	
the calling process,	-1 of wait gets blocked	semWait (blocks)	
0			semWait
	0	unblocks	semSignal
		does work	does work

```
void A() {
    semSignal (sA);
    semWait (sB);
    /*do what A does*/;}
void B() {
    semWait (sA);
    semSignal (sB);
    /*do what B does*/;}
semaphore sA = 0;
semaphore sB = 0;
void main() {
    parbegin (A(), B());}
```

Make A and B always run together!

When B comes first

sA	sB	A	B
0	0		
-1			semWait (blocks)
0		semSignal	unblocks
	-1	semWait (blocks)	
	0	unblocks	semSignal
		does work	does work

```
void A() {
    semSignal (sA);
    semWait (sB);
    /*do what A does*/;}
void B() {
    semWait (sA);
    semSignal (sB);
    /*do what B does*/;}
semaphore sA = 0;
semaphore sB = 0;
void main() {
    parbegin (A(), B());}
```

Make A and B always run together!

When A and B run in parallel

sA	sB	A	B
0	0		
1		semSignal	
0			semWait
	-1	semWait (blocks)	
	0	unblocks	semSignal
		does work	does work

```
void A() {
    semSignal (sA);
    semWait (sB);
    /*do what A does*/;}
```

```
void B() {
    semWait (sA);
    semSignal (sB);
    /*do what B does*/;}
```

```
semaphore sA = 0;
semaphore sB = 0;
void main() {
    parbegin (A(), B());}
```

Now consider three processes A, B and C…

```
void main() {
    parbegin (A(), B(), C());
}
```

```
void A() {
    /*do what A does*/;
}
```

```
void B() {
    /*do what B does*/;
}
```

```
void C() {
    /*do what C does*/;
}
```

Make C run before A and B

```
semaphore syn = 0;
void main() {
    parbegin (A(), B(), C());
}
```

```
void A() {
    semWait (syn);
    /*do what A does*/;
}
```

```
void B() {
    semWait (syn);
    /*do what B does*/;
}
```

```
void C() {
    /*do what C does*/;
    semSignal (syn);
    semSignal (syn);}
```

to make any process run without being blocked, add n1 semsignals for that

Make C run before A and B

Arrival order: A, B, C

syn	A	B	C
0			
-1	semWait (blocks)		
-2		semWait (blocks)	
			does work
-1	unblocks		semSignal
0	does work	unblocks	semSignal
		does work	

```
void A() {
    semWait (syn);
    /*do what A does*/;
}
```

```
void B() {
    semWait (syn);
    /*do what B does*/;
}
```

```
void C() {
    /*do what C does*/;
    semSignal (syn);
    semSignal (syn);}
```

Make C run before A and B

Arrival order: A, C, B

syn	A	B	C
0			
-1	semWait (blocks)		
			does work
0	unblocks		semSignal
1	does work		semSignal
0		semWait	
		does work	

```
void A() {
    semWait (syn);
    /*do what A does*/;
}
```

```
void B() {
    semWait (syn);
    /*do what B does*/;
}
```

```
void C() {
    /*do what C does*/;
    semSignal (syn);
    semSignal (syn);}
```

Make C run before A and B

Arrival order: B, A, C

syn	A	B	C
0			
-1		semWait (blocks)	
-2	semWait (blocks)		
			does work
-1		unblocks	semSignal
0	unblocks	does work	semSignal
	does work		

```
void A() {
    semWait (syn);
    /*do what A does*/;
}
```

```
void B() {
    semWait (syn);
    /*do what B does*/;
}
```

```
void C() {
    /*do what C does*/;
    semSignal (syn);
    semSignal (syn);}
```

Make C run before A and B

Arrival order: B, C, A

syn	A	B	C
0			
-1		semWait (blocks)	
			does work
0		unblocks	semSignal
1		does work	semSignal
0	semWait		
	does work		

```
void A() {
    semWait (syn);
    /*do what A does*/;
}
```

```
void B() {
    semWait (syn);
    /*do what B does*/;
}
```

```
void C() {
    /*do what C does*/;
    semSignal (syn);
    semSignal (syn);}
```

Make C run before A and B

Arrival order: C, A, B

syn	A	B	C
0			does work
1			semSignal
2			semSignal
1	semWait		
0	does work	semWait	
		does work	

```
void A() {
    semWait (syn);
    /*do what A does*/;
}
```

```
void B() {
    semWait (syn);
    /*do what B does*/;
}
```

```
void C() {
    /*do what C does*/;
    semSignal (syn);
    semSignal (syn);}
```

Make C run before A and B

Arrival order: C, B, A

syn	A	B	C
0			does work
1			semSignal
2			semSignal
1		semWait	
0	semWait	does work	
	does work		

```
void A() {
    semWait (syn);
    /*do what A does*/;
}
```

```
void B() {
    semWait (syn);
    /*do what B does*/;
}
```

```
void C() {
    /*do what C does*/;
    semSignal (syn);
    semSignal (syn);}
```

Make C run before A and B Using 2 semaphores

```
semaphore sA = 0;
semaphore sB = 0;
void main() {
    parbegin (A(), B(), C());}
```

```
void A() {
    semWait (sA);
    /*do what A does*/;
}
```

```
void B() {
    semWait (sB);
    /*do what B does*/;
}
```

```
void C() {
    /*do what C does*/;
    semSignal (sA);
    semSignal (sB);}
```

We enforce additional order: A will always run before B. This is not required!!

WRONG!!!

when we want any one to execute at first, we implement one semaphore, but when we want a strict order we implement two semaphores

a gets signal first and starts executing

If you have followed me so far…

1. Make C run after both A and B.
2. Make the three processes run in this order: B, A, C.

Make C run before A and B

```
int x;
void main() {
    parbegin (A(), B(), C());
}
```

```
void A() {
    /*do what A does*/;
    x = x + 2;
}
```

```
void B() {
    /*do what B does*/;
    x = x – 5;
}
```

```
void C() {
    /*do what C does*/;
    x = x – 1;
}
```

Make C run before A and B

```
semaphore syn = 0;
semaphore me = 1;
int x;
void main() {
    parbegin (A(), B(), C()); }
```

```
void C() {
    /*do what C does*/;
    x = x – 1;
    semSignal (syn);
    semSignal (syn); }
```

```
void A() {
    semWait (syn);
    /*do what A does*/;
    semWait (me);
    x = x + 2;
    semSignal (me); }
```

```
void B() {
    semWait (syn);
    /*do what B does*/;
    semWait (me);
    x = x – 5;
    semSignal (me); }
```

Make C run before A and B

```
int x;
void main() {
    parbegin (A(), B(), C());
}
```

```
void A() {
    /*do what A does*/;
    print(x + 2);
}
```

```
void B() {
    /*do what B does*/;
    print(x – 5);
}
```

```
void C() {
    /*do what C does*/;
    x = x – 1;
}
```

Lessons learned

- We explored binary semaphores.
- We implemented mutual exclusion using semaphores.
- We implemented synchronization using semaphores.

Things to do

Read the book.

Write your questions on a piece of paper and keep it in front of you during the online session.

If you have suggestions relevant to content or quality of this lecture, email them to me.